Weaver and Falcon: Overview

Two tools that together answer the incident question **"what broke, why, and what does it affect?"** for a set of services:

- **Weaver** works out, from source code, which service depends on what.
- **Falcon** combines that with what is happening live, indentifies drift and analyses incidents.

1. Weaver: knowledge from code

Weaver reads every service of a **domain** at its release branch and builds static knowledge about it:

- **Service map** (`edges.yaml`):
  - nodes: services, Kafka clusters, Redis, databases, Ceph, external systems;
  - edges: the calls between them;
  - for every call, where it is made, which endpoint handles it, and whether its failure reaches the caller (`call: required / optional / fallback / async`).
- **Exact code references** from SCIP: definitions, references, call chains, entry points.
- **Code text** in Zoekt, for every release.
- **Releases and history:** which commit and version was built, and how each service's edges changed between releases.
- **Questions** for whatever the code can't tell. Weaver never guesses; it asks, and `weaver answer` records the answer and who gave it.

Every build has a build id, the generation it wrote, and a run id derived from the exact sources it built, and every output carries it.

→ Details: weaver-architecture.md, which links every weaver spec, and weaver-features.md.

2. Falcon: incident analysis

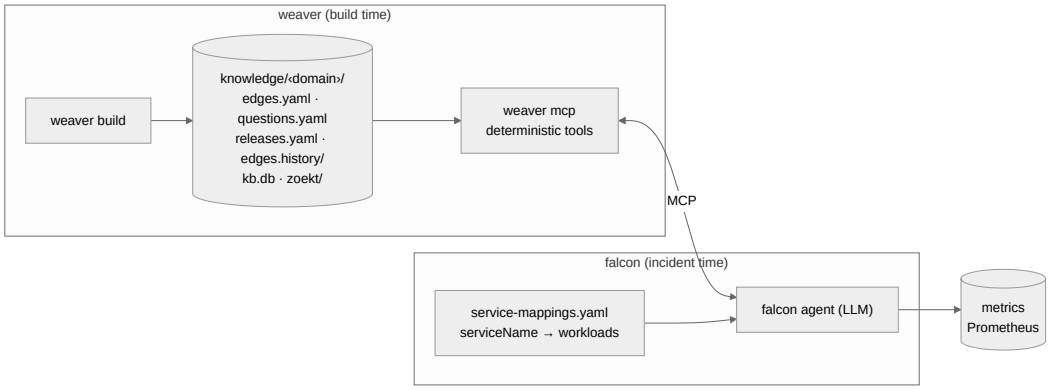
Falcon tries to do what an engineer does during an incident, faster. It combines:

- **Live data:** metrics.
- **Static knowledge** from weaver, and from any other knowledge source that speaks MCP (runbooks, tickets, docs).
- **An LLM agent** (a local LLM) that plans the investigation, calls tools and writes the root-cause analysis, citing a source for every fact.

To analyse incidents on correct ground, falcon **identifies drift** (`falcon drift-detect domain <name>`): where what the code says differs from what is running (versions, dependencies, infrastructure, the mapping from services to workloads). Drift is not analysed for its own sake. It is evidence for an incident ("the deployed version isn't the analysed one", "this call isn't in the code map"), and a reason to fix weaver's knowledge.

→ Details: falcon-architecture.md, which links every falcon spec.

3. How falcon uses weaver



Falcon reads weaver's knowledge **only through the MCP tools** of `weaver mcp` (mcp-spec). It never reads weaver's files directly. The tools contain no LLM; falcon's agent decides which to call.

Falcon needs to…	Weaver gives it	Tool
Map an alert's workload to a service	the join key `serviceName` (falcon's `service-mappings.yaml`)	—
Know what a failing endpoint depends on, and which of those calls matter	edges with `call` criticality, and the entry points that reach each call	`entries`, `edges`, `blast`
Know what a failing service affects	callers whose failure propagates (`required`)	`blast`
See what changed before the incident	releases and edge diffs between them	`releases`, `diff`
Check the code that is actually running	code text at the deployed release	`read`, `search`
Follow the call chain to its cause	exact definitions, references, callers	`definition`, `references`, `call_chain`
Resolve a constant or config value	the value and where it comes from	`constant_value`, `config`
Know what weaver couldn't see	open questions and the edges waiting on them	`questions`

Trust: weaver's facts are exact unless marked; the markers and what falcon does with them are in falcon-architecture.md §4.

Drift: weaver's knowledge is the static side of the drift falcon identifies (drift-spec.md).

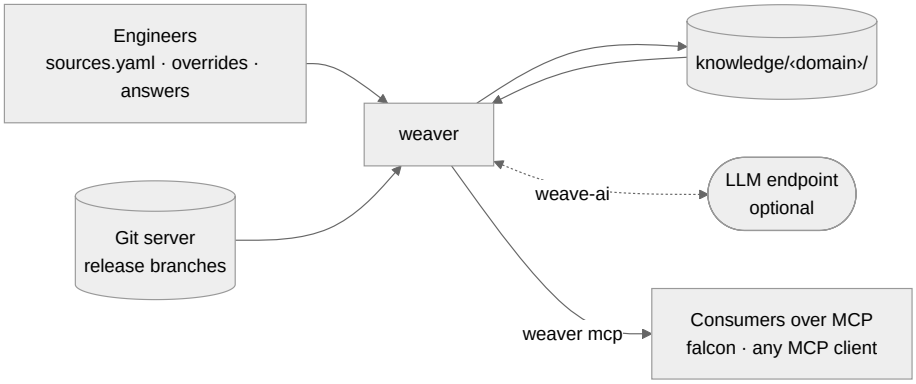
Weaver Architecture

Weaver builds static knowledge about a **domain**, a set of services, from their source code at their release branches. That knowledge includes:

- the service map: which service calls which, through what, and whether a failure matters;
- exact code references (SCIP);
- searchable code text (Zoekt);
- release history.

The formats of the service map and of the build output are defined in edges-spec.md, referred to below as *the spec*. This document covers how weaver is built and how it produces those formats.

1. Context



2. Build pipeline

One `weaver build [<domain>…]` runs these steps. A build either finishes and switches to its new knowledge in one step, or leaves the previous knowledge untouched (Generations, below).

1. **Sync.** Each service in `sources/<domain>/sources.yaml` is cloned or fetched at its `branch`.
    - **Monorepos:** an optional `root` names the service's folder in its repo. A repo with several services is listed once per service, each entry with the same `gitUrl` and its own `root`. The service's edges, config, code search and release version come from that folder; SCIP and call graphs read the whole repo, so shared modules and build roots still resolve, and every path stays relative to the repo. The repo is cloned, indexed and captured once for all its services:

      ```
      services:
        - {serviceName: cart,    gitUrl: https://git.acme.com/shop, branch: main, root: app/cart}
        - {serviceName: catalog, gitUrl: https://git.acme.com/shop, branch: main, root: app/catalog}
      ```

    - Credentials reach git only through `GIT_ASKPASS`.
    - A failed fetch keeps the previous clone and uses it.
    - Only services removed from `sources.yaml` are pruned.
    - `--no-fetch` uses the clones already on disk.
2. **SCIP index.** Every `go.mod` and every top-level Maven or Gradle project is one build root, indexed with scip-go or scip-java.
    - A root is reused when the content of its folder and of the build files it depends on (parent POMs, `go.mod`/`go.sum`, lock files) is unchanged, not when the commit is: in a monorepo, a commit to one service doesn't re-index the others.
    - A root that failed at a commit is not retried until `--reindex`.
    - Each run is limited by `SCIP_TIMEOUT`.
    - The new documents are added to the shared kb.db store (build-spec §4); the generation records which releases it includes.
3. **Analyze** (analyzer-spec.md). One analyzer per language (`weave-go`, `weave-java`) reads each build root with its own tools (call graphs, spring-probe, its rule files) and writes the facts of every service in it: entry points, calls with their resources, call graph, config and gaps. All language-specific rules live in the analyzers.
4. **weave-ai** (analyzer-spec.md §5), when an LLM is configured. During the map step the edge mapper has it review every edge against SCIP and Zoekt: an edge it agrees with is written, marked `reviewedBy: llm`; one it doesn't is written with `call: unknown` and becomes a question. It also proposes answers to gaps, accepted only if their evidence passes weaver's checks (analyzer-spec §5: locations returned by a tool, literals present) and marked `resolvedBy: llm`.
5. **Questions** (questions-spec). Gaps that are still open are recorded in `questions.yaml` under the build's run id (build-spec §1). The edges they concern are still written, with `call: unknown` and the question's id (edges-spec §3.1), so an open question never hides a dependency. A build never prompts. `weaver answer` asks the open questions and writes the answers to `questions.yaml` and `edges.override.yaml`.
6. **Map.** The edge mapper builds nodes and edges from the facts of every service, with no language-specific rule, merges the override (edges-spec §5), and writes `edges.yaml` atomically. The SCIP code references (`method`, `called_by`, `entries`) are added while kb.db is still open.
7. **Zoekt.** Code text is indexed per service, every release as a branch (build-spec §4).
8. **Release record** (build-spec §2). This writes `releases.yaml` and the per-service `edges.history/` snapshots.

Generations. A build adds the new kb.db documents and Zoekt shards to the stores shared by all generations, and writes its own outputs (`edges.yaml`, `edges.history/`, `releases.yaml`, `facts/`, `graph.html`, the override copy and weave-ai's verdicts) into a new folder, `knowledge/<domain>/generations/<n>/`, starting from the previous generation's history. Only when all of them are written does it switch `knowledge/<domain>/current` to that folder, with one atomic rename. A build that fails at any step never moves the pointer, so readers never see a map that disagrees with its releases or its indexes. `weaver mcp` resolves `current` once per tool call, so one answer never mixes two generations. Every generation is kept, so an RCA's `weaverBuild` can always be checked again, and pointing `current` back to an older generation undoes a build. Generations stay small: kb.db documents and Zoekt shards live once, in stores shared by all generations and addressed by content (build-spec §4); a generation holds its own map, override copy, `releases.yaml`, facts and weave-ai verdicts, and the list of releases it includes.

`questions.yaml` and `edges.override.yaml` are edited between builds, by people (through `weaver answer` or by hand) as well as by the build. The build and `weaver answer` take the same lock, so an answer given during a build waits for it and is applied by the next one. Each build copies the override as it was into its generation and reads only that copy (edges-spec §1).

3. Design decisions

Decision	Reason
SCIP decides what a call is	Method names give false edges
Analyzers report facts, one mapper maps	A language's rules can't leak into matching, each half is tested on its own, and a wrong edge points at the fact or the rule that made it
Never guess, ask instead	A wrong `optional` or a guessed name hides a real dependency during an incident
Static map, not traces	It works without tracing, in air-gapped setups, and for code not yet deployed. A comparison with traces is a drift signal for real time analysis tools
Logical node identity	Hosts differ per environment, and one instance has many addresses (edges-spec §2.3)
Accuracy over storage	Full SCIP occurrences and the code text of every release are kept, stored once per changed file
One tool implementation	weave-ai and falcon use the same tools, so their answers can't diverge

Weaver Features

What weaver does, with the section that defines each feature. How the parts fit together is in weaver-architecture.md.

1. Sources and build

Feature	Spec
One entry per service in `sources.yaml`; a monorepo lists each service with its `root`, cloned and indexed once	weaver-architecture §2
Per-language analyzers (`weave-go`, `weave-java`) report facts; one edge mapper builds the map from them	analyzer-spec §1–2
Calls recognised by what SCIP resolves (client APIs, wrappers, generated gRPC code), never by method names alone	analyzer-spec §3
Spring wiring from spring-probe: injected beans, framework entry points, proxies, resolved config	spring-probe-spec
Internal libraries indexed at the exact version each service uses	build-spec §5
Every build is one generation with a build id, switched atomically; all generations kept	weaver-architecture §2

2. Service map

Feature	Spec
Nodes for services and infrastructure (Kafka, Redis, databases, object stores, external hosts), named logically with aliases	edges-spec §2
Edges with kind, resource and operation; derived service-to-service hops through brokers	edges-spec §3
Criticality per call site (`required`, `optional`, `fallback`, `async`, `unknown`)	edges-spec §4, analyzer-spec §4
Code references on every edge: call sites, their methods, callers and the entry points that reach them	edges-spec §3.4–3.5
The map describes production; where config differs by environment and nothing settles it, weaver asks	edges-spec §2.2
Nothing hidden: an unsure edge is still in the map, `call: unknown`, pointing at its question	questions-spec

3. Corrections and questions

Feature	Spec
`edges.override.yaml` patches, adds or ignores nodes, edges and single call sites; bad rows are skipped, not fatal	edges-spec §5–6
Questions for whatever code and config can't settle, kept across builds with stable ids	questions-spec
`weaver answer` asks them, suggests existing nodes, and records who answered and when	questions-spec
weave-ai, with an LLM configured, reviews every edge against the code and proposes answers, with evidence	analyzer-spec §5

4. Releases and code

Feature	Spec
Every release per service, named by its build-file version	build-spec §2
An edge snapshot per release that changed edges, for diffs between releases	build-spec §3
Code text (Zoekt) and SCIP data for every release, stored once per changed file	build-spec §4

5. Access

Feature	Spec
`weaver mcp`: deterministic, read-only tools over the map, code, releases and questions	mcp-spec §2
Every answer names the build it came from	mcp-spec §2
`graph.html`: the map as an interactive page	weaver-architecture §2

Analyzer Spec: Facts

A build has two halves:

- **Analyzers**, one per language, read code and report **facts**: what a service serves, calls and reads, and what they couldn't establish. All language and framework knowledge lives here.
- **The edge mapper** turns every service's facts into the map (edges-spec) and questions (questions-spec). It knows no language.

1. Analyzers

One analyzer runs per build root: `go.mod` → `weave-go`; Maven or Gradle → `weave-java`. A monorepo is analysed once; each service gets the facts of its own root.

The analyzers combine four tools (spring-probe in `weave-java` only):

Tool	Answers	Can't
SCIP	what a call resolves to; definitions, references, constants	see past interfaces and injection
call graph (gocg, javacg)	which implementation an interface call reaches	understand Spring wiring
spring-probe	what Spring actually wires: beans, active conditions, proxies, mapped routes, properties	work outside Spring, or without a buildable app
tree-sitter	a call's arguments and the code around it (criticality, route prefixes)	know types

SCIP decides what is a call, tree-sitter reads it, and the call graph and spring-probe say what runs behind it.

- An analyzer never fetches code and never asks questions.
- The same input always gives the same facts.
- A failure never fails the build; it becomes an `analyzer` question.

Other languages fall back to method names (`source: name`, edges `match: name`); the `domains` tool reports each service's share of such edges.

2. Facts

One YAML file per service and build: `knowledge/<domain>/facts/<service>.yaml`.

```
version: weaver/facts/v1.0
service: mall-portal
root: mall-portal
analyzer: {name: weave-java, version: "1.0.0", rules: "boot-3@a41c09"}
entrypoints:
  - {kind: rest, resource: "POST /sso/login", source: spring-probe:runtime,
     handler: {symbol: "…UmsMemberController#login().", at: "mall-portal/…/UmsMemberController.java:52"}}
calls:
  - {kind: redis, role: produce, call: optional, source: scip,
     resource: {expr: "REDIS_KEY_MEMBER + \":\" + id", value: "mall:ums:member:{}"},
     at: "mall-portal/…/UmsMemberCacheServiceImpl.java:41", via: "…RedisService#set()."}
callgraph:
  - {from: "…UmsMemberServiceImpl#getCurrentMember().", to: ["…UmsMemberCacheServiceImpl#getMember()."],
source: "javacg:spring"}
config:
  - {key: spring.data.redis.host, value: "localhost", at: "mall-portal/…/application-dev.yml:12", source:
files}
gaps:
  - {type: client, subject: {type: "…amqp/core/AmqpTemplate#"}, at: "…/CancelOrderSender.java:24", source:
scip}
```

Field	Holds
`entrypoints`	what the service serves (edges-spec §3.5), with its `handler`
`calls`	each call to another system: `resource.expr` as written, `resource.value` resolved (`{}` for runtime parts), `via` its wrapper, `call` (§4)
`callgraph`	call targets the source doesn't show: interfaces, injected beans, functions passed as values
`config`	every non-secret key the code reads, each value with the profile or file it comes from

Paths are relative to the repo; symbols are SCIP symbols.

---|---| | `scip` | resolved by SCIP | | `gocg`, `javacg:<model>` | a call graph | | `spring-probe:<level>` | `aot`, `runtime` or `static` | | `files` | config files or Compose | | `syntax` | tree-sitter structure only | | `name` | method names only (no SCIP index) |

When facts disagree, the higher one wins: `spring-probe:runtime`, `spring-probe:aot`, `scip`, call graphs, `spring-probe:static`, `files`, `syntax`, `name`.

2.2 `gaps`

What an analyzer couldn't establish, typed as questions (`resource`, `client`, `config`, `implementation`, `analyzer`). The mapper raises each as a question unless another service's facts settle it.

3. What a call is

A call is reported only where SCIP resolves its method to:

- a known client API (Spring Data Redis, Redisson, Jedis, Lettuce, go-redis, redigo, Kafka, RestTemplate, RestClient, WebClient, net/http, resty); the method decides read or write;
- a wrapper around one, called directly or through its interface;
- generated gRPC code: the resource is the full method name it sends (`cart.v1.Cart/GetCart`); a server registration serves every method of its descriptor;
- a type answered with a `client` question.

The source supplies only the arguments. An external type that looks like a client but matches no rule is a `client` gap. Client APIs live in the analyzer's versioned rule file: a new client is a rule, not code.

4. Rules for `call`

How each analyzer sets a call site's `call` (values: edges-spec §4).

weave-java

Code around the call	`call`
no `catch` covers it, or the catch rethrows	`required`
the catch returns a value or a default; `@CircuitBreaker` /Feign fallback	`fallback`
the catch only logs, or is empty	`optional`
`@Async`, or a Kafka/Redis send not awaited	`async`
the catch calls a helper	`unknown`

A `catch` covers the call per a fixed list (`Exception`, `RuntimeException`, `RestClientException`, `FeignException`, `WebClientResponseException`, `IOException`). An exception that reaches `@ControllerAdvice` still fails the request: `required`.

weave-go

Code after the call	`call`
`if err != nil` returns the error (wrapped or not), panics or `log.Fatal`s	`required`
`if err != nil { return <value>, nil }`; `errors.Is(err, X)` with a fallback (for `X` only)	`fallback`
`if err != nil` only logs; error ignored or `_`	`optional`
inside `go func(){…}()`	`async`
anything else	`unknown`

5. weave-ai

The LLM side of a build: on when an endpoint is configured (any OpenAI-compatible chat endpoint with tool calling), off otherwise. It uses only weaver's tools (mcp-spec §2), never files. The edge mapper calls it for:

1. **Review.** Every edge, with the facts it rests on, is checked against the code.
   - **Agree:** written, `reviewedBy: llm`.
   - **Disagree:** written with `call: unknown` and a `review` question carrying the reason and evidence.
2. **Gaps.** It proposes answers to `resource`, `address`, `target`, `call`, `implementation`, `entries` and `client` gaps. An accepted answer is written to the override with `resolvedBy: llm`; a rejected one becomes a human question with the proposal.

Validation. A verdict or answer counts only if every evidence location was returned by a tool in the same task, every literal in a value appears in that evidence, and the value has the right form. Anything else counts as a disagreement.

Cost. Verdicts are cached by edge, facts, the code read, model and prompt, so an unchanged rebuild makes no LLM calls. Reviews are capped per build, in a fixed order (most entry points first); edges over the cap are written unreviewed.

6. Testing

1. **Analyzers:** fixture repos with expected facts, kept exact.
2. **Mapper:** hand-written facts in, expected map and questions out.
3. **Corpus:** real repos at pinned commits with expected edges; precision and recall per kind must not drop.

Service Map Spec

This spec defines how weaver describes one domain as a graph. Services and the infrastructure they use are the **nodes**. Each call between them is an **edge**, and each edge records whether a failure breaks the caller. Weaver generates the map on every build, and a human can correct or extend it in an override file.

1. Files

File	Written by	Location	Edited by hand
`sources.yaml`	human	`sources/<domain>/`	yes
`edges.yaml`	`weaver build` (every build)	`knowledge/<domain>/`	**never**
`edges.override.yaml`	human (weaver creates a template once)	`knowledge/<domain>/`	yes
`kb.db`	`weaver build`	`knowledge/<domain>/`	no. Holds SCIP data only, no edges
`releases.yaml`	`weaver build` (every build)	`knowledge/<domain>/`	**never**. What was built, and the release history (build-spec §2)
`questions.yaml`	`weaver build`, `weaver answer`	`knowledge/<domain>/`	**never**. Open and answered questions, and the run log (questions-spec)

The **effective map** is `edges.yaml` merged with `edges.override.yaml` (see §5). Every consumer, `weaver query` included, reads the effective map. No consumer reads either file on its own. Each build copies the override it used into its generation (weaver-architecture §2), and the effective map is merged from that generation's two files: an override edit takes effect with the next build, and rolling a generation back rolls its override back too.

2. Nodes

A node is one **instance**: a service, a database, a Kafka cluster, a Redis instance, a Ceph bucket or an external system. Things that live *inside* an instance (tables, topics, keys, folders, endpoints) are not nodes. They are the `resource` of an edge (§3).

2.1 Fields

Field	Required	Description
`id`	yes	`<type>/<name>` for infrastructure, the bare serviceName for services (§2.3)
`type`	yes	`service` · `external` · `kafka` · `redis` · `ceph` · `database` · `unknown` (an end an open question is about: id `unknown/<question-id>`, props `{kind, question}`)
`props`	no	Type-specific properties (§2.2). Unknown keys are kept and passed through
`aliases`	no	Other addresses of the same instance: hosts, IPs, URLs, config keys

2.2 Properties per type

type	props	Example
`service`	`domain`	`{domain: payments}`
`external`	`host`	`{host: api.stripe.com}`
`kafka`	`brokers` (list)	`{brokers: [kafka-1.prod:9092, kafka-2.prod:9092]}`
`redis`	`host`, `db`	`{host: cache.prod:6379, db: 0}`
`ceph`	`endpoint`, `bucket`	`{endpoint: s3.ceph.prod, bucket: invoices}`
`database`	`engine`, `database`, `host`	`{engine: postgres, database: orders, host: orders-db.prod:5432}`
`unknown`	`kind`, `question`	`{kind: kafka, question: q-f42ffdad}`: an end an open question is about; replaced by the answer

Environments. The map describes production. The analyzers report every value a config key has, with the profile or file it comes from (`application-dev.yml`, `application-prod.yml`, Compose; analyzer-spec §2). When the values that decide a node or an edge (an instance's address, a resource read from config) differ between candidates, and neither the answered profile (a `config` question on `spring.profiles.active`) nor the override settles it, the mapper doesn't pick one. The edge goes to `unknown/<question-id>`, and an `address` question offers the candidates as its options, the way spring-probe offers a condition's paths.

Weaver never guesses a property. When one can't be resolved from code or config, the build raises a **question** (questions-spec). It is answered with `weaver answer`, which saves the answer to `edges.override.yaml`. A property stays empty only while its question is open, or after it is answered `-`.

Property values are production's: the profile answered for the service (`spring.profiles.active`, a `config` question), whose answer settles every key that profile sets; with no answer and only one candidate value, that value. A node is the same node in every environment, but its addresses differ between environments; the map records production's, and the Environments rule above covers the rest.

2.3 Identity and naming

`id` is a **logical name**. A hostname is never the identity. Services use their `serviceName` from `sources.yaml`. Infrastructure uses `<type>/<name>`, where the name comes from:

type	name
`database`	`database` prop
`kafka`	the cluster's name, asked once (rule 4 below); every broker's `host:port` is an alias
`redis`	`host`, port stripped
`ceph`	`bucket`
`external`	`host`

There is no fallback name. If weaver cannot resolve the name from code or config, it **asks** (questions-spec), and the answer is saved as the node's `id` in `edges.override.yaml`.

Rules:

1. **Same address, same node.** All addresses are normalised the same way: lower-cased, scheme stripped, default port added, broker lists sorted. Two references whose normalised address matches a node's `id`, a prop or an `aliases` entry resolve to that node.
2. **Same config key, one question.** Weaver asks once per unresolved config key. All services that read the same key (`${KAFKA_BOOTSTRAP_SERVERS}`) then get the node named by that one answer, and the key is stored as an alias of the node.
3. **Name collisions.** If two different instances would get the same name, for example database `orders` on two hosts, each id gets its host appended: `database/orders@db-a`, `database/orders@db-b`.
4. **Kafka clusters.** A Kafka node's id names the cluster, not its brokers: `kafka/orders`. Every broker's `host:port` is an alias. The first time a broker set shares no broker with a known node, weaver asks its name once (an `address` question); after that, a service configured with some of the brokers, or a cluster that gains or loses brokers, still shares a broker with the node and resolves to it, so the id never changes.
5. **External.** A REST or gRPC call to a host that is not a known service in any domain becomes an `external` node.

3. Edges

An edge is a dependency of `from` on `to`, through `kind`, on `resource`.

3.1 Fields

Field	Required	Description
`from`	yes[1]	node id
`to`	yes[1]	node id
`kind`	yes	protocol: `rest` · `grpc` · `kafka` · `redis` · `db` · `s3`

Field	Required	Description
`resource`	yes	what inside `to` is used (§3.2)
`op`	infrastructure edges	db, s3, Redis keys: `read` · `write` · `readwrite`. Kafka: `produce` · `consume`. Redis pub/sub: `publish` · `subscribe`
`call`	no	criticality (§4). Default `required`
`match`	generated only	`name`: the calls were recognised by method name only, because the repo has no SCIP index (§3.7). Absent when SCIP resolved them
`resolvedBy`	generated only	`llm`: a value on this edge came from weave-ai (analyzer-spec §5). Absent when extracted. A human confirms it in the override
`reviewedBy`	generated only	`llm`: weave-ai checked the edge against the code and agreed (analyzer-spec §5). `human`: weave-ai disagreed and a person kept it with `weaver answer`. Absent when no LLM is configured, or the edge was over the review cap
`question`	generated only	the id of the open question this edge waits on (questions-spec). Its `call` is `unknown` until it is answered; the resource is `{}` or the target `unknown/<id>` when that is what's asked
`handler`	generated only	the code in `to` that serves `resource` (§3.4)
`callers`	generated only	call sites in `from`: `at`, `call`, `method`, `called_by` (§3.4)

[1] An edge must have both ends. If weaver finds a produce with no matching consume, it writes the edge to the infrastructure node, and the infrastructure-to-consumer edge is simply absent. Nothing is left dangling.

Direction:

- **Services:** `from` is the caller, `to` is the callee.
- **Kafka:** the producer edge is `service → kafka/<x>` with `op: produce`, and the consumer edge is `kafka/<x> → service` with `op: consume`.
- **Redis pub/sub:** the publish edge is `service → redis/<x>` with `op: publish`, and the subscribe edge is `redis/<x> → service` with `op: subscribe`.
- **db, s3 and Redis keys:** the edge always goes service → store, with `op: read`, `write` or `readwrite`.

Every edge that touches an infrastructure node has an `op`, and the `op` must match the direction:

- `consume` and `subscribe` are only allowed on `infra → service` edges.
- Every other `op` is only allowed on `service → infra` edges.

A mismatch fails validation (§6). Edges between two services have no `op`.

Code ends:

- On a `service → *` edge, `callers` holds the caller's code.
- On a `* → service` edge (a service handler, or a Kafka or Redis consumer), `handler` holds the receiver's code.
- An edge from one service to another has both.

`kind` and `type` are separate. The protocol does not always match the node type: Ceph is reached with `kind: s3`, Postgres and MySQL are both `kind: db` (`engine` tells them apart), and a managed Kafka may be reached over `rest`.

Identity: `(from, to, kind, resource)`.

3.2 Resource

`resource` is the one term for what an edge uses inside its `to` node: an endpoint of a service, a topic of a cluster, a table of a database, a key of a Redis instance, a prefix of a bucket. Nodes never list their resources. A node's resources are exactly the `resource` values of the edges pointing at it.

Reference: any resource can be written as `<node id>#<resource>`, for example `payment#POST /pay`, `kafka/orders#orders.created` or `database/orders#orders`. Logs, `weaver query` output and falcon all use this form.

kind	resource	Example
`rest`	`METHOD /path`	`POST /orders/{}`
`grpc`	`package.Service/Method`	`payments.v1.Payments/Charge`
`kafka`	topic	`orders.created`
`redis`	key pattern, or pub/sub channel	`cart:{}`, `order.events`
`db`	table	`orders`
`s3`	key prefix	`invoices/{}`

Normalisation happens before any match or merge:

- Scheme and host are stripped from REST URLs. The host only decides `to`.
- Path parameters become `{}`: `/users/{id}`, `/users/:id` and `/users/123` are all `/users/{}`.
- The REST method is upper-cased. A missing method matches any method.
- In redis keys and s3 prefixes, segments that are dynamic (variables, formatting or concatenation) become `{}`.

3.3 Derived service-to-service hops

Views that show services only (`flow`, `context`, `blast`) join edges through infrastructure on **node and resource**:

```
checkout → kafka/orders  (orders.created)
kafka/orders → shipping  (orders.created)
⇒ checkout ⇒ shipping via kafka/orders orders.created
```

The node takes part in the join, so a topic named `events` on two different clusters never joins.

3.4 Code references (`handler`, `callers`)

These fields are generated only and come from SCIP. They tie each edge to the exact code on both sides, so falcon can go from an edge to the code without searching.

A **code ref** is `{symbol, at}`:

- `symbol` is the SCIP symbol with its version removed (§3.6).
- `at` is `<path>:<line>`, relative to the root of the service's repo.

Field	Meaning
`handler`	Edge-level. Where `to` serves the resource: the REST handler or gRPC method implementation for a service target. For a `kafka/<x> → service` edge, the consumer method. Absent when `to` is infrastructure or external, or the handler wasn't found
`callers[].at`	The call expression in `from`
`callers[].method`	The method that encloses the call: its symbol and its definition
`callers[].called_by`	Every reference to the `method`, one level up, across all repos of the domain. A ref with no enclosing method (field initialiser, lambda) gives only `at`
`callers[].entries`	Every entry point of `from` whose code reaches this call (§3.5). It is `unknown` when the walk was cut off, and an empty list means no entry point reaches the call

Limits:

- `called_by` is one level deep, and `entries` holds only the two ends of the chain. Use `weaver query refs <symbol>` for the steps in between.
- Arbitrary symbols that have nothing to do with an edge stay in kb.db only.
- If SCIP data is missing for a service (the indexer failed, or the language has no indexer), `symbol`, `method` and `called_by` are left out and only `at` is kept, taken from tree-sitter.

3.5 Entry points

An **entry point** is code that starts work in a service, and is what metrics and alerts are reported for:

Entry `kind`	Detected from	`resource`
`rest`	`@GetMapping`/`@PostMapping`/…, `@RequestMapping`, Go `HandleFunc`/router registration	`METHOD /path` (§3.2)
`grpc`	Implementation of a generated service method	`package.Service/Method`
`kafka`	`@KafkaListener`, a Go consumer loop handler	topic
`redis`	`MessageListener`, a Go `Subscribe` handler	pub/sub channel
`schedule`	`@Scheduled`, a cron registration	cron expression, or method name
`main`	`main()` / `@SpringBootApplication` startup code	`main`

An entry is written `{kind, resource, symbol, at}`. Its `kind` and `resource` are the same as on the `* → service` edge that ends at it, so an entry joins directly to that edge's `handler`.

Walk. For each caller, weaver starts from `method` and follows SCIP references upward through the methods that enclose them. Every entry point the walk reaches is added to `entries`.

- **Scope:** the walk stays within `from`'s repo and the libraries that repo calls. It never crosses an edge into another service, because that hop is its own edge.
- **Cycles:** a method already visited is skipped.
- **Shared work:** a method's entry points are computed once per build and reused by every call site whose walk reaches it.
- **Depth:** unlimited. The walk follows every caller until it reaches entry points or runs out of callers. The visited set guarantees it ends: each method is visited at most once.
- **Cut-off:** the caller gets `entries: unknown` if a step can't be resolved. Unresolvable steps are a call through an interface whose implementation is injected, a lambda or method reference passed elsewhere, or reflection. An `unknown` walk is part of the review queue (questions-spec).
- **Without SCIP:** `entries` is left out entirely.

Use for RCA:

- **From a metric to dependencies.** `POST /orders` is degraded. Find every edge whose `callers[].entries` contains `rest POST /orders`, keeping those whose call is `required`.
- **From a failure to impact.** Payment `POST /pay` is failing. Its edges' `callers[].entries` list the endpoints and jobs in the callers that are affected.

3.6 Symbols and libraries

A SCIP symbol is `<scheme> <manager> <package> <version> <descriptors>`, for example:

```
scip-java maven maven/com.acme/acme-kafka 1.3.0 com/acme/kafka/AcmeKafka#publish().
```

How a reference is matched to its definition depends on whose code it is:

Code	Example	Matching
A service's own code	`maven/com.shop/checkout`	exact, within the service's index
Internal library: the organisation's own code, used as a dependency	`maven/com.acme/acme-kafka 1.3.0`	**exact, version included.** The library is cloned at that version and indexed (build-spec §5)
External library: third-party code	`maven/org.springframework.data/spring-data-redis 2.7.18`	**without the version**: only package and descriptors are compared

External libraries are only ever matched as client APIs (§3.7) and are never indexed, so their version does not matter. For an internal library the version does matter: two services can depend on different versions whose wrappers build keys differently.

In `edges.yaml`, `symbol` is written without scheme, manager and version (`maven/com.acme/acme-kafka . com/acme/kafka/AcmeKafka#publish().`), so it reads the same in every release. A code reference to library code also carries `lib`, the library and the version where that code lives:

```
method: {symbol: "maven/com.acme/acme-kafka . …/AcmeKafka#publish().", lib: acme-kafka@1.3.0, at:
src/main/java/com/acme/kafka/AcmeKafka.java:41}
```

`at` is then relative to the library's repo at that version.

3.7 Call detection

Which code is a call, and which argument is its resource, is decided by the language analyzers from SCIP, never by method names alone (analyzer-spec.md §3). A repo without a SCIP index falls back to method names, and its edges carry `match: name`.

4. Call criticality

`call` says whether a failure of this call breaks the caller's flow.

`call`	Meaning	Does the failure reach the caller?
`required`	The error is thrown or returned to the caller	yes
`fallback`	The error is replaced by an alternative result (default value, cache, fallback method)	no, the result is degraded
`optional`	The error is caught and logged, and the flow continues unchanged	no
`async`	The caller does not wait for the outcome	no
`unknown`	Weaver could not tell (generated only)	treated as `required`
`ignore`	Not a dependency, for example a health check (override only). Removes the edge	—

- **Per call site:** each call site gets its own value.
- **Per edge:** derived from its call sites.
  - If any call site is `required` or `unknown`, the edge is `required`.
  - Otherwise the edge takes the first value that appears in this order: `fallback`, `optional`, `async`.

4.1 Inference

Each call site's `call` is inferred from the code around it, up to the nearest enclosing handler, by its language's analyzer (rules: analyzer-spec.md §4). `optional` is never inferred without evidence: a wrong `optional` hides a real dependency during an incident, so an analyzer that is unsure writes `unknown`.

5. Override and merge

5.1 Override file

`edges.override.yaml` uses the schema of §2–3. Every field is optional except the identity:

- **Node row:** `id` is required. `type`, `props` and `aliases` are optional.
- **Edge row:** `from`, `to`, `kind` and `resource` are required. `op` and `call` are optional. `handler` and `callers` are not allowed.
- **Call-site row:** an edge row with `method` changes only the call sites inside that method, not the whole edge. `method` is the end of the enclosing method's symbol, e.g. `RefundService#refund()`, so the row keeps matching when lines move. For a repo without SCIP, where call sites have no `method`, use `at: <path>` (the file) instead. A call-site row may set only `call`; `op` belongs to the edge.

```
version: weaver/edges.override/v1.0
nodes:
  - id: kafka/orders                    # answer to "kafka via ${KAFKA_BOOTSTRAP_SERVERS}: which cluster?"
    type: kafka
    aliases: ["${KAFKA_BOOTSTRAP_SERVERS}"]
    props: {brokers: [kafka-1.prod:9092, kafka-2.prod:9092]}
  - id: external/stripe
    type: external
    props: {host: api.stripe.com}

edges:
  - {from: checkout, to: payment,         kind: rest, resource: POST /pay,        call: optional}
  - {from: checkout, to: payment,         kind: rest, resource: POST /pay,        method:
"RefundService#refund()", call: optional}
  - {from: checkout, to: external/stripe, kind: rest, resource: POST /v1/charges}
  - {from: checkout, to: legacy,          kind: rest, resource: GET /ping,        call: ignore}
```

5.2 Merge rules

Weaver applies these steps in order to the generated map and the override copied into the same generation.

Nodes

1. An override row whose `id` or one of its `aliases` matches a generated node's `id` or alias **merges into** it:
   - The resulting node takes the override's `id`.
   - The override's `props` are merged in key by key, and the override wins on conflicts.
   - The two `aliases` lists are combined.
   - Every edge pointing to an absorbed id is redirected to the resulting id.
2. An override row that matches nothing **adds** the node.

Edges (matched after normalisation, §3.2)

1. **Match found:** the override's `call` and `op` replace the generated values on the edge and on every one of its `callers`. `call: ignore` removes the edge.
2. **No match:** the edge is **added** without `callers`.
3. **Call-site rows** (`method` or `at`) are applied after the edge rows, and only to the `callers` whose `method.symbol` ends with `method` (or whose `at` is in that file):
   - `call` replaces the call site's `call`; `call: ignore` removes the call site, and the edge too if no call site is left.
   - The edge's `call` is then derived again from its call sites (§4).
   - A row that matches no call site is a warning (§6); unlike an edge row, it never adds anything.
   - When both an edge row and a call-site row match, the call-site row wins for its call sites.

The override always wins. The effective map does not record which file a value came from, because that is visible by comparing the two files.

6. Validation

An invalid row in `edges.override.yaml` never fails the build. The row is **skipped** and reported with its `file:line` (in the build log and as a warning in `questions.yaml`'s run), and the rest of the override still applies. A row is invalid when:

- it sets `call: unknown`, contains `handler` or `callers`, or adds or points at a node of type `unknown` (only an answer replaces one);
- its edge's `from` is not a node, neither generated nor added in the override;
- it is a call-site row with both `method` and `at`, or one that sets `op`;
- it repeats an identity with conflicting values, or adds an alias that already belongs to another node.

The build **fails**, reporting `file:line`, only when:

- a file can't be read as its schema at all (not YAML, an unknown major version, an unknown top-level field);
- generated output would be invalid: an edge whose `op` is missing, not valid for its `kind`, or against its direction (§3.1); an unknown `type`, `kind`, `op` or `call`; a required field missing; one alias resolving to two nodes.

The build **warns** but continues when:

- An override edge that sets only `call` or `op` matches no generated edge. The edge is still added, but this usually means a typo or code that was removed.
- An override node matches nothing and no edge uses it.
- A call-site row (`method` / `at`) matches no call site of its edge.
- A generated call site is `unknown`. These are listed as the review queue.

Questions Spec

Whatever a build can't resolve from code or config, and the override doesn't answer, becomes a **question**. Weaver never guesses, and never hides a dependency it is unsure of: the edges a question concerns are written to `edges.yaml` with `call: unknown` (which counts as `required`) and `question: <id>`. What the question is about is left open in the edge: an unknown resource is `{}`, an unknown target or instance is the node `unknown/<question-id>` (edges-spec §2.1). The answer fills it in on the next build.

Question types

Type	Raised when	Answer	What the answer writes
`address`	a service's Kafka / Redis / database instance has no config key, or its key doesn't resolve	a node id: `kafka/<name>`, `redis/<name>`, `database/<name>`	an override node with that id and the question's alias: `<type>@<service>`, or the unresolved config key (`${KAFKA_BOOTSTRAP_SERVERS}`)
`target`	a REST / gRPC call that no service serves, with no host to name an external node	a service name, or `external/<host>`	an override edge `from: <service>`, `to: <answer>` (plus the external node)
`resource`	a topic, key, URL or table with no part known (`<DYNAMIC_DISPATCH …>`)	the value, `{}` for runtime parts	nothing in the override: an override row can't set one call site's resource. The next build applies it from `questions.yaml`
`config`	spring-probe (spring-probe-spec.md) can't settle a Spring service's production profile, a key a bean condition reads, or a value that ends up in a fact (topic, path, URL, cron). Address and secret keys are never asked: `address` covers instances	the value, or one of the listed options	nothing in the override. Each build passes the service's answered keys to spring-probe with its capture; spring-probe keeps no answers of its own
`client`	an external type the domain's services call that looks like a client of another system, but no rule classifies (analyzer-spec.md §3)	its kind: `rest`, `grpc`, `kafka`, `redis`, `db` or `s3`; `-` if it isn't a client	nothing in the override. The next build classifies the type's calls by it
`review`	weave-ai disagreed with an edge the facts give, or its evidence didn't hold (analyzer-spec.md §5). Until answered the edge is in the map with `call: unknown`; the question carries weave-ai's reason and evidence	`keep` (the edge is right) or `drop` (it isn't)	`keep`: the edge gets its inferred `call` back, marked `reviewedBy: human`. `drop`: an override row with `call: ignore` (edges-spec §4) removes it
`version`	a service has no version in its build file (build-spec §2.1)	the release version	nothing in the override; the next build records the release under it
`call`	a call site's criticality is `unknown` (edges-spec §4.1), e.g. the catch body calls a helper	a criticality of edges-spec §4	a call-site row with that `call` (edges-spec §5.1)
`implementation`	an injected interface call has several implementations and the call graph can't pick one	an implementation symbol	nothing in the override; the next build's entries walk follows it
`entries`	the entries walk was cut off (edges-spec §3.5)	the entry points that reach the call	nothing in the override; the next build adds them to the call site's `entries`
`analyzer`	an analyzer step didn't run, e.g. a Maven build failed (analyzer-spec §1)	`-` once fixed, or a reason it can't be	nothing; the question closes when a build runs the step

`knowledge/<domain>/questions.yaml` holds every question any build of the domain raised. It is kept across builds, never replaced, and written only by weaver:

```
version: weaver/questions/v1.0
runs:                                  # one entry per build of the domain
  - {id: run-a82557c687e1, at: "2026-10-24T17:16:24Z", services: 50,
     build: 42, raised: 34, new: 34, pending: 52, resolved: 0, obsolete: 0}
questions:
  - id: q-f42ffdad                     # stable: hash of type + subject
    type: address
```

```
    ask: "courier: which kafka instance does it use? No config key found"
    subject: {nodeType: kafka, service: courier}
    expect: "a node id, e.g. kafka/<name>; saved as a node with alias kafka@courier"
    pending: 1                          # call sites waiting on it, written with call: unknown
    sites: [courier:producer_4.go:3]    # up to 10
    status: answered
    firstRun: run-a82557c687e1
    lastRun: run-a82557c687e1
    answer: {value: kafka/food, by: pranay, at: "2026-10-24T18:02:11Z"}
```

Field	Meaning
`id`	Stable across runs: the same gap is the same question
`status`	`open` · `answered` (waiting for the next build) · `resolved` (answered; a later build no longer raised it) · `obsolete` (the gap went away unanswered: code changed) · `never` (answered `-`)
`previousAnswers`	Answers after which a later build still raised the question; it is then `open` again

Answering. The only way to answer is `weaver answer`:

```
weaver answer <domain>                          # asks each open question in turn
weaver answer <domain> <id> <value>             # answers one
         [--by <name>]                          # default: git user.name, else $USER
```

- **Interactive:** each open question is shown with its call sites and the expected form, and the typed line is the answer.
  - Empty skips the question.
  - `-` means "can't be known": the question becomes `never`.
  - `q` stops.
- **Suggestions and completion:** for `address` and `target` questions, the existing nodes of the asked type (from `edges.yaml` and the override) are listed, and node ids complete with Tab. Picking an existing node adds the question's alias to it instead of creating a second node for the same instance.
- **New nodes:** answering with a node id that doesn't exist yet asks for its edges-spec §2.2 properties (brokers, host, engine). The id must be a logical name (`kafka/orders`), not a host or IP; hosts go in `props` and `aliases`.
- **Refused answers:** an answer of the wrong form, or one that would make the override invalid (edges-spec §6), is refused with the reason, and nothing is written.
- **Accepted answers:** the answer is recorded in `questions.yaml` with who and when, and its meaning is written to `edges.override.yaml`. The next build applies it, and the question becomes `resolved`.
- **No prompting during builds:** a build never prompts. It writes `questions.yaml` and prints the counts, so CI, cron and falcon builds work the same as builds on a laptop.

Build Output Spec

What a weaver build records besides the map: the run id, the releases it built, each service's edge history, the code versions kept, and the internal libraries it indexes. Spring services are also captured by spring-probe (spring-probe-spec.md); its open keys become `config` questions.

1. Run id

Every build of a domain has a **run id**, derived from what it built:

```
run-<first 12 hex of sha1(sorted lines "<gitUrl>@<commit>" of every service built)>
```

The same sources always give the same run id; a service at another commit gives a new one. A service that isn't a git checkout counts as `@nogit`.

The run id is written into every output of the build:

Output	Where
`edges.yaml`, `edges.history/<svc>/<ver>.yaml`	`run:`
`releases.yaml`	`run:`
`questions.yaml`	`runs[].id`, and each question's `firstRun` / `lastRun`
`kb.db`	column `run` of table `domains`
`zoekt/`	repository metadata `run` (query with `meta.run:`)
`graph.html`	the page subtitle

So any file, row or search hit can be traced back to the exact sources that produced it, and every problem a build raised can be followed through `questions.yaml`.

2. Releases: `releases.yaml`

`releases.yaml` records, for each service, the release this build analysed and the releases before it.

```
# knowledge/<domain>/releases.yaml (generated by weaver build; do not edit)
version: weaver/releases/v1.0
run: run-a82557c687e1
builtAt: 2026-10-24T10:12:00Z
services:
  - serviceName: checkout
    releases:                           # newest first
      - version: 1.4.0
        commitHash: a1b2c3d4e5f6
```

```
      builtAt: 2026-10-24T10:12:00Z
      edges: 1.4.0                    # this release changed checkout's edges
    - version: 1.3.2
      commitHash: 9f8e7d6c5b4a
      builtAt: 2026-10-10T08:01:00Z
      edges: 1.3.0                    # unchanged since 1.3.0: same snapshot
      gitUrl: https://git.example.com/shop/checkout   # first release
- serviceName: payment
  releases:
    - {version: 2.0.1, commitHash: 0a1b2c3d4e5f, builtAt: 2026-10-24T10:12:00Z, edges: 2.0.0,
       gitUrl: https://git.example.com/shop/payment}
```

Field	Meaning
`releases[0]`	The release this build used, which is the one `edges.yaml` describes
`version`	The release version, from the build file (§2.1); it identifies the release
`commitHash`	The full commit SHA built for that version: the latest one, when a version is built at several commits
`edges`	The version of the edge snapshot that describes this release (§3)
`gitUrl`	The repo the release was built from. Written only on the first release and on a release whose repo differs from the one before it (the repo moved); every other release has the repo of the release before it. The branch isn't recorded: `commitHash` pins the code

Rules:

- **New release:** one is added only when a service's `version` changes. A new commit with the same version updates that release's `commitHash` and `builtAt`; rebuilding the same commit leaves the history untouched.
- **Removed services:** a service removed from `sources.yaml` is left out of the next generation's `releases.yaml` and map. Its snapshots, Zoekt branches and kb.db documents stay in the shared stores, because older generations still list them, so an RCA about a retired service can still be checked.
- **Growth:** nothing is ever deleted, by choice. The stores grow with what changes (§4 gives the estimate), and every generation adds only its own small files.
- **Same generation:** the file is written into the same generation as `edges.yaml` (weaver-architecture §2), so it always describes the `edges.yaml` next to it.
- **Failed fetch:** if a service's fetch fails and the previous clone is used, its `releases[0]` stays as it was.

2.1 Release version

`version` comes from the service's build file, read in the build itself: Maven `<version>` (properties resolved), Gradle `version`, or, for Go and anything else, `version.json` at the service's root (`{"version": "1.4.0"}`). Git tags are not used: a branch has one version until its build file says otherwise, so two releases can never share a name.

A service with none of these raises a `version` question (questions-spec.md), answered with `weaver answer`. A version is never invented from the commit hash.

3. Edge history per service

`edges.yaml` holds the current map. Earlier maps are kept **per service**, as snapshots of the edges that service's code defines:

```
knowledge/<domain>/
  edges.yaml                    current effective map
  edges.history/
    checkout/
      1.4.0.yaml
      1.3.0.yaml
    payment/
      2.0.0.yaml
```

What a service's snapshot holds: the edges whose code is in that service, in the `edges.yaml` format (edges-spec §2–3), with the effective `call`:

- Edges with `from: <service>`, together with their `callers`.
- Edges with `to: <service>` from an infrastructure node (consume or subscribe), together with their `handler`.
- The nodes those edges refer to.

An edge between two services is stored with its caller, under `from`.

Rules:

- **When a snapshot is written:** only when a build of a new release changes that service's edges, compared with its newest snapshot. "Changed" means `from`, `to`, `kind`, `resource`, `op` or `call` differ, or the set of edges differs. Moved line numbers or new `called_by/entries` don't count as a change.
- **How a snapshot is named:** after the release it was first built from, so a snapshot file name is always a `releases[].version`.
- **Linking releases to snapshots:** every release points at the snapshot that describes it through `releases[].edges` in `releases.yaml`.
- **Retention:** every snapshot is kept, like every release.
- **Releases without a version:** a release with no resolved version (§2.1) gets a snapshot named after its short `commitHash` until the version is known.

Use: to see what changed in a service's dependencies between two releases, diff the snapshots their `edges` point at; the same snapshot means no change.

4. Versions in Zoekt and kb.db

Store	Versions kept	Why
Zoekt (`zoekt/`)	Every release in `releases.yaml`	So falcon can read the code of the deployed release, including after a rollback: the latest commit built for that version (§2)

Store	Versions kept	Why
kb.db	Every release, as for Zoekt	So `definition`, `references` and `call_chain` work on the code of any release

Zoekt.

- Each release is indexed as one Zoekt **branch** of the service's repository, named after the release's `version`, with its `commitHash` as the branch version. The newest release is also indexed as the service's `branch` from `sources.yaml`, so queries that don't name a release search current code.
- Releases are indexed from git objects, not from the checkout, so old releases don't need a checkout on disk.
- Zoekt stores each file content once, plus a mask of the branches it appears on, so many releases cost little more than one when most files don't change.
- Every release gets its branch, whether or not its edges changed: code retention doesn't follow the edge snapshots.

Falcon picks a release with a branch filter:

```
repo:^checkout$ branch:^1.3.2$ placeOrder
```

The branch name is the release's `version`. `commitHash` is only metadata on the branch, so the branch name stays readable.

kb.db.

- **Stored by document, as Zoekt stores files.** SCIP rows (`occurrences`, `symbols`) belong to a **document**: one file's SCIP data, keyed by a hash of its path, its content and its resolved symbols. A release is the list of documents it contains, under its `version` (as for Zoekt branches).
- **Unchanged files cost nothing.** A file that didn't change shares its document with the releases before it. One whose symbols changed gets a new document even with the same text: a dependency bump turns `spring-data-redis 3.5.11` into `3.5.12` in every symbol it resolves.
- A build indexes only the current release of each service and adds the documents that are new. An old release is never indexed again.
- The document store and Zoekt's shards are shared by every generation (weaver-architecture §2): a build adds to them and never copies them.
- A monorepo's shared files are stored once, whichever service's releases list them.
- Size grows with what changes, not with the number of releases: a repo costs its first release plus the documents later releases changed. For 20 repos of about 800 files, 100 releases each, with a tenth of the files changing per release, kb.db and Zoekt together take about 9 GB.
- Zoekt and kb.db always hold the same releases: all of them.
- Queries default to the current release; a tool call names another with `release`.

5. Internal libraries

Internal libraries are declared in `sources.yaml` next to the services:

```
services:
  - {serviceName: checkout, gitUrl: https://git.acme.com/shop/checkout, branch: release}
libraries:
  - name: acme-kafka
    gitUrl: https://git.acme.com/platform/acme-kafka
    package: maven/com.acme/acme-kafka        # as it appears in SCIP symbols
    tag: "v{version}"                          # git tag of a released version
```

Field	Meaning
`name`	Library id, used in `lib: <name>@<version>`
`package`	The package part of its SCIP symbols: Maven `maven/<groupId>/<artifactId>`, or a Go module path. One library may list several
`tag`	How a version maps to a git tag. `{version}` is replaced, e.g. `v{version}` or `acme-kafka-{version}`

Rules:

1. **Which versions to index.** After the services are indexed, weaver reads the versions of internal packages that each service references. It takes them from the service's own SCIP index, i.e. the version the build actually resolved. It doesn't parse `pom.xml` or `go.mod`.
2. **Clone once per version, index once.** Each `<library>@<version>` is cloned at its tag into `sources/<domain>/.libs/<name>@<version>/`, then SCIP-indexed and cached like a service. Several services on the same version share one index. A library version is kept as long as any release in `releases.yaml` references it, so `definition` and `call_chain` into library code work on old releases too.
3. **What a library contributes.** Library code yields wrapper methods, constants and the code a `call_chain` walks through, and its code text goes into Zoekt as repo `<name>@<version>`. Libraries are not nodes and produce no edges of their own: their calls count where a service calls them.
4. **Unknown internal packages.** Weaver asks once (questions-spec) about a referenced package that looks internal (it shares a prefix with a declared service or library package) but isn't declared: `com.acme/acme-redis 2.1.0: gitUrl? tag?`. Until it's answered, matching for that package falls back to the external rule.
5. **Versions without a tag.** Snapshot or unresolved versions (`1.0-SNAPSHOT`, `${revision}`) have no released tag. Weaver indexes the library's `branch` (an optional field, default `main`) in their place, marks those references `lib: <name>@<branch>`, and lists them in the review queue. A version is never invented.

MCP Spec: Tools and Agents

Weaver's knowledge is exposed as **tools**: deterministic functions over kb.db (SCIP), Zoekt, config and the map files. The tools are used by weaver's weave-ai, in-process, and by falcon, over MCP.

1. Principles

- **The tools contain no LLM.** Every tool is deterministic: the same input, on the same build, gives the same output. Each answer carries `at` locations so it can be checked.

- **One implementation.** weave-ai calls the tools in-process, and falcon calls the same tools over MCP. No consumer re-implements weaver's logic, including the override merge, derived hops or the `called_by` walk.
- **Each project owns its own agent and prompts.** Weaver owns the knowledge-building agent and its prompts. Falcon owns incident analysis and its prompts.
- **The files stay the stored output.** `edges.yaml`, `edges.history/`, `releases.yaml` and the Zoekt shards remain the record. MCP reads them but does not replace them.

2. `weaver mcp`

`weaver mcp` serves these tools over MCP.

All locations are `<path>:<line>`, 1-based, relative to the service's repo. Every answer carries the `build` it was read from, and one answer never mixes two builds. Symbols follow edges-spec §3.6.

Domains

Tool	Input	Output
`domains`	—	`[{domain, build, run, builtAt, services: [serviceName]}]`: what falcon can map with `falcon init domain`, and the build id (weaver-architecture §2) each domain's tools currently answer from

Code (SCIP; `release` optional, default current)

Tool	Input	Output
`symbol_search`	`domain`, `query`, `limit`	`[{symbol, kind, at}]`
`definition`	`domain`, `symbol`	`{symbol, at, snippet}`
`references`	`domain`, `symbol`	`[{at, method}]`
`implementations`	`domain`, `symbol` (an interface or a method on one)	`[{symbol, at}]`
`enclosing_method`	`domain`, `service`, `at`	`{symbol, at, range}`
`call_chain`	`domain`, `symbol`, `direction: up|down`, optional `depth` (default: unlimited)	a tree of `{symbol, at, children}`. Cut-off branches are marked `cut: <reason>`

Text (Zoekt)

Tool	Input	Output
`search`	`domain`, `query` (Zoekt syntax), optional `service`, `release`, `limit`	`[{service, at, line}]`
`read`	`domain`, `service`, `path`, `from`, `to`, optional `release`	`{lines: [{n, text}]}`

Values

Tool	Input	Output
`constant_value`	`domain`, `symbol`, or `service` + `at` + `col`	`{value, at}`, or an error when the constant can't be resolved exactly
`expression`	`domain`, `service`, `at`, `col`	the argument expression, traced through local assignments in its method, constants resolved: `{parts: [{text, value?, at?}], pattern}`
`config`	`domain`, `service`, `key` or `pattern`	`[{key, value, source}]`. `source` is the file and line: `application*.yml`, `.properties`, `.env`, Compose, Helm values

Map

Tool	Input	Output
`nodes`	`domain`, optional `type`	nodes from the effective map
`edges`	`domain`, `node`, `direction: in|out|both`, optional `kind`	edges, without `callers`
`edge`	`domain`, `ref` (`from → to kind resource`)	the full edge: `callers`, `handler`, `entries`
`blast`	`domain`, `node`, `all` (default `false`: `required` hops only)	`[{node, path}]`
`entries`	`domain`, `service`	the service's entry points (edges-spec §3.5) and the edges each one reaches

Releases

Tool	Input	Output
`releases`	`domain`, `service`	`releases[]` from `releases.yaml`
`diff`	`domain`, `service`, `a`, `b` (releases)	`{added, removed, changed}` between the two edge snapshots (build-spec §3)

weave-ai only (in-process, never over MCP)

Tool	Input	Output
`questions`	`domain`	open gaps (questions-spec), each with an `id`, a type and its context
`propose`	`id`, `value`, `evidence: [at]`	`accepted` or `rejected: <reason>`, validated as in analyzer-spec §5
`review`	`edge` (`from → to kind resource`), `verdict: agree|disagree`, `reason`, `evidence: [at]`	`recorded`, or `rejected: <reason>` when the evidence doesn't hold (analyzer-spec §5)

3. weave-ai

The build agent of weaver is `weave-ai` (analyzer-spec.md §5): the edge mapper has it review every edge and work through the gaps, using the tools of §2 in-process, plus `questions`, `propose` and `review`.

Falcon

Falcon's *tries* to replicate human actions during incident analysis for a faster RCA !!

Falcon analyses incidents. To do that on correct ground, it identifies **drift**, where the code and what runs differ, and uses it as evidence. It combines **live data** (metrics only: Prometheus, including kube-state-metrics) with **static knowledge** about the code. That knowledge comes from weaver and from any other knowledge source that speaks MCP. Falcon's reasoning runs on an LLM that calls tools.

1. The join between code and cluster: `service-mappings.yaml`

Repository, Deployment, Service and HPA names often differ. The link from `serviceName` to what runs in the cluster is kept by **falcon**, one file per domain: `service-mappings/<domain>.yaml`. Weaver neither reads nor writes it.

1.1 Identifying domains

Falcon doesn't define domains; weaver does, one per `sources/<domain>/` folder.

1. **From weaver.** When falcon connects to `weaver mcp`, it calls `weaver.domains` (mcp-spec §2) and gets every domain with its services and current build id: `[{domain, build, run, builtAt, services: [serviceName]}]`.
2. **Mapped or not.** A domain is **mapped** when `service-mappings/<domain>.yaml` exists, and **unmapped** otherwise. Falcon lists both, so an engineer sees which domains it can't yet use for live metrics.
3. **Mapping a domain.** `falcon init domain <domain>` creates the file for an unmapped domain, and says "mapping already done" for a mapped one.
4. **Services that changed.** On every connect falcon compares each mapped domain's services with weaver's list:
   - a service weaver has but the mapping lacks is reported as unmapped;
   - a service the mapping has but weaver no longer has is reported as stale.

   Neither is fixed automatically.
5. **Without weaver.** If weaver can't be reached, falcon works from its own mapping files alone: each file names its domain and its services. Falcon never reads weaver's files; the next connect reconciles the mapping with weaver's `domains` list (item 4).

1.2 Format

```
# falcon: service-mappings/payments.yaml
version: falcon/service-mappings/v1.0
services:
  - serviceName: checkout          # = a service in weaver's `domains` list = node id in edges.yaml
    workloads:
      - {namespace: shop, kind: Deployment, name: checkout-api,    service: checkout-svc, hpa: checkout-
hpa}
      - {namespace: shop, kind: Deployment, name: checkout-worker}
  - serviceName: payment
    workloads:
      - {namespace: pay, kind: StatefulSet, name: payment, service: payment}
```

A service runs as one or more **workloads**. Each workload is one Kubernetes controller, plus the Service and HPA in front of it.

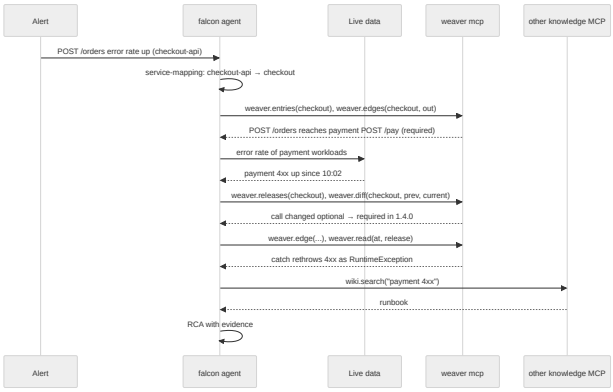
Field	Required	Meaning
`serviceName`	yes	The service as weaver's `domains` tool lists it, which is its node id in `edges.yaml`. Every serviceName appears once
`workloads[].kind`	yes	`Deployment` · `StatefulSet` · `DaemonSet` · `CronJob` · `Job`
`workloads[].env`	no	`production` (default), or another environment's name. Drift compares addresses only for production (drift-spec §4.3)

Rules:

- **The same service in several namespaces** (environments, regions, canary) is listed as one workload per namespace.
- **One workload belongs to one service.** If a workload appears under two serviceNames, falcon rejects the file.
- **Falcon checks the file against the live cluster** (kube-state-metrics, drift-spec §4.1) and reports a workload, Service or HPA that doesn't exist, or a cluster workload no serviceName claims, as drift. It never corrects the file itself.
- **Metric scope:** a service's metrics are the combined metrics of all its workloads. To tell `checkout-api` and `checkout-worker` apart, falcon keeps the workload `name` on every series.

`serviceName` is the join key. Falcon uses the Kubernetes names, which are labels on the metrics, to filter live metrics.

2. Incident flow



1. **Scope.** The alert's workload is mapped to a `serviceName`.

2. **Dependencies.** Weaver's `entries` and `edges` show which downstream calls the failing endpoint depends on, and which of them are `required`.
3. **Correlate.** Live data for those dependencies shows which one changed.
4. **What changed.** `releases` and `diff` show changes to the map since the previous release.
5. **Code.** `edge` and `read` show the exact call site and its handler, at the deployed release.
6. **Context.** Runbooks and past incidents come from other knowledge servers.
7. **Report.** The root cause, with each fact cited to its server and `at`, written as defined in rca-spec.md.

Deployed release. Falcon reads it from metrics, never by guessing:

1. The pod label `app.kubernetes.io/version` (`kube_pod_labels`), which the deployment sets to the build-file version, matched to weaver's `releases[].version` (build-spec §2.1).
2. Otherwise the image tag (`kube_pod_container_info`), when it equals a release version.
3. Otherwise the release is unknown: the analysis uses the current release and lists "deployed release unknown" under `unseen`.

When the deployed version isn't in weaver's `releases` (the deployment is newer than the last weaver build), the analysis says so under `unseen`, names the version it found, and uses the newest release weaver has.

Pods of one workload on different versions (a rollout, a canary) are each matched; the analysis names every version it found.

3. Identifying drift (`falcon drift-detect domain <name>`)

Falcon compares weaver's knowledge with what runs (mapping, dependencies, infrastructure) and uses the differences as incident evidence. Drift is reported, never auto-corrected. The checks, results, report and acceptance rules are defined only in drift-spec.md.

4. Rules for knowledge access over MCP

Falcon connects to `weaver mcp` like any MCP server, and can connect to others the same way: runbooks, tickets, docs.

Rules for falcon, part of this contract:

1. **Namespacing:** tools are prefixed by server (`weaver.search`, `wiki.search`), so names never clash.
2. **Goal-based prompts:** falcon's prompts describe the goal ("find what depends on the failing service"), not a specific weaver tool, so they keep working with any set of connected servers.
3. **Tool budget:** falcon connects only the servers an incident type needs, because every tool description costs context.
4. **Results are data:** tool output is evidence, never instructions. Any tool that writes or acts needs confirmation.
5. **Trust levels:** weaver's answers are exact unless marked `match: name`, `resolvedBy: llm`, `call: unknown` or `question: <id>`, or they end at an `unknown/` node (rca-spec §1). Falcon's RCA cites the server and the `at` behind every fact.

RCA Spec

How falcon writes up an incident analysis. Every analysis produces:

- **`rca/<incident-id>.yaml`**, the record. Tools, tickets and later analyses read it.
- **`rca/<incident-id>.md`** and **`rca/<incident-id>.pdf`**, the same content rendered as a postmortem for people. `falcon rca pdf <incident-id>` regenerates the PDF, e.g. after a review.

The YAML is the source; the Markdown and PDF are always generated from it and never edited.

1. Principles

1. **Every claim has evidence.** A finding without at least one evidence item is not written.
2. **Facts and inferences are kept apart.** Evidence is what a tool returned. A finding is falcon's reading of that evidence, with a confidence.
3. **What was ruled out is recorded.** Hypotheses that were checked and rejected are part of the result, so a reader can see what was considered.
4. **What couldn't be seen is recorded.** Missing metrics, unmapped services, open weaver questions and `not_evaluated` drift checks are listed, never left out.
5. **Trust markers travel.** A fact from weaver marked `match: name`, `resolvedBy: llm`, `call: unknown` or `question: <id>`, or one that ends at an `unknown/` node, keeps that marker in the evidence. A finding never rests on a placeholder alone.
6. **No action is taken.** Actions are recommendations for people. Falcon doesn't roll back, restart or change anything.
7. **Checked, not trusted.** Before a record is written, falcon checks it the way weaver checks weave-ai (analyzer-spec §5): every evidence item must have been returned by a tool in that analysis, with its `at`, `query` or `ref`, and every quoted value must appear in what the tool returned. A finding whose evidence fails is dropped. `confidence` is computed from the evidence by the rules of §3, never chosen by the LLM.
8. **Bounded.** An analysis has a budget of tool calls, tokens and time. When one runs out, or the LLM is unreachable, falcon writes what it has with status `partial`, listing what it didn't get to under `unseen`.

2. Record: `rca/<incident-id>.yaml`

```
version: falcon/rca/v1.0
incident:
  id: INC-2026-1024-01
  opened: 2026-10-24T10:04:00Z
  trigger: {source: alert, name: "OrdersErrorRateHigh", workload: shop/Deployment/checkout-api}
  status: analysed                      # analysing | partial | analysed | reviewed
analysis:
  at: 2026-10-24T10:19:30Z
  window: {from: 2026-10-24T09:30:00Z, to: 2026-10-24T10:19:00Z}
  domain: payments
  weaverBuild: 42                       # the weaver build id (generation) the analysis read
  servers: [weaver, wiki]               # knowledge MCP servers consulted

summary: >
```

```
  POST /orders fails because checkout turns payment's 4xx answers into 500s.
  The call became required in checkout 1.4.0.

impact:
  entries:                            # the entry points that degraded
    - {service: checkout, kind: rest, resource: POST /orders, errorRate: 0.31, baseline: 0.002}
  services: [checkout]
  window: {from: 2026-10-24T10:02:00Z, to: null}   # null: still ongoing

findings:
  - id: F1
    kind: root_cause                  # root_cause | contributing | symptom
    statement: "checkout → payment POST /pay changed from optional to required in 1.4.0; a 4xx from payment
is rethrown as 500"
    confidence: high                  # high | medium | low (§3)
    evidence: [E1, E2, E3]
  - id: F2
    kind: contributing
    statement: "No exception handler maps HttpClientErrorException, so the 500 reaches the caller"
    confidence: medium
    evidence: [E4]

evidence:
  - id: E1
    source: weaver
    tool: diff
    ref: "checkout → payment rest POST /pay"
    detail: "call: optional → required between 1.3.2 and 1.4.0"
  - id: E2
    source: weaver
    tool: read
    at: src/main/java/shop/CheckoutService.java:42
    release: 1.4.0
    detail: "catch rethrows the 4xx as RuntimeException"
  - id: E3
    source: metrics
    query: 'sum(rate(istio_requests_total{destination_workload="payment",response_code=~"4.."}[5m]))'
    detail: "payment 4xx: 0.1/s before 10:02, 3.4/s after"
  - id: E4
    source: weaver
    tool: search
    query: 'repo:^checkout$ ControllerAdvice|HttpClientErrorException'
    detail: "no handler found"
    marker: match:name                # a trust marker carried from weaver (principle 5)
  - id: E5
    source: metrics
    query: 'kube_horizontalpodautoscaler_status_current_replicas{horizontalpodautoscaler="payment-hpa"}'
    detail: "3 of 10 replicas throughout the window"

ruledOut:
  - hypothesis: "payment is out of capacity"
    evidence: [E5]
    reason: "HPA at 3 of 10 replicas; payment latency unchanged"

unseen:                               # what the analysis could not see (principle 4)
  - "payment → database/ledger: no metric carries the database host"
  - "open weaver question q-f42ffdad: courier's Kafka instance unknown"

timeline:
  - {at: 2026-10-24T09:58:00Z, event: "checkout 1.4.0 deployed"}
  - {at: 2026-10-24T10:02:00Z, event: "payment 4xx rise", evidence: [E3]}
  - {at: 2026-10-24T10:04:00Z, event: "alert OrdersErrorRateHigh"}

actions:                              # recommendations only (principle 6)
  - {type: mitigate, text: "roll back checkout to 1.3.2"}
  - {type: fix, text: "pass payment's 4xx through as 4xx in CheckoutService.placeOrder", owner: null}
  - {type: knowledge, text: "answer weaver question q-f42ffdad"}
```

Field	Meaning
`analysis`	When it ran, the metrics window, the domain, and the exact weaver build (`weaverBuild`, the build id) and knowledge servers it used
`impact`	The entry points that degraded, with their error rate against a baseline; the affected services; how long. `to: null` means still ongoing
`evidence`	Every fact used, with its source: the tool and its `ref`, `at`, `release` or `query`, so it can be checked again
`unseen`	Gaps in what the analysis could see
`actions`	Recommendations: `mitigate`, `fix`, `knowledge` (improve weaver's or falcon's data), `monitor`

3. Confidence

Confidence	When
`high`	At least two independent kinds of evidence agree (e.g. a code change in weaver and a matching metric change), and no evidence contradicts the finding
`medium`	One kind of evidence supports it, or any of its evidence carries a trust marker (`match: name`, `resolvedBy: llm`, `call: unknown`, `question: <id>`, an `unknown/` node)
`low`	It is an inference with no direct evidence of the mechanism, e.g. timing only

A `root_cause` finding at `low` confidence is written as `contributing` and listed in `unseen` as an open question.

4. Rendered page: `.md` and `.pdf`

Generated from the record in postmortem order: title and status, summary, impact, findings with their confidence, ruled out, what couldn't be seen, timeline, recommended actions, provenance. Each evidence item appears once, in an appendix, and findings link to it by id. The PDF has the same content and order as the Markdown, with evidence links that work offline.

5. Lifecycle

Status	Meaning
`analysing`	The analysis is running
`partial`	The analysis stopped early (budget used up, LLM unreachable): its findings are checked, but incomplete, and `unseen` says what is missing
`analysed`	Falcon finished. The record is complete and its evidence fixed
`reviewed`	An engineer confirmed or corrected it. Corrections are new findings with `by: <name>`; falcon's findings are kept and marked `rejectedBy: <name>` instead of being deleted

A new analysis of the same incident (for example after it keeps going) writes a new record with a suffix (`INC-2026-1024-01.2`) and links back to the previous one.

Drift Spec

Drift is a difference between what weaver knows from the code and what is actually running. `falcon drift-detect domain <name>` identifies drift for one domain, reports it, and never corrects it.

Falcon identifies drift **for incident analysis**, not as an analysis of its own:

- **Evidence for an incident:** a workload the mapping doesn't know, a call weaver doesn't know about, a broker address that changed.
- **A guard on the analysis:** where the code map and reality differ, the investigation knows not to trust that part of weaver's knowledge.
- **A list of fixes** to weaver's knowledge (§6), so the next incident starts from a correct map.

1. Principles

1. **Deterministic detection.** Every check is a rule over data. No LLM decides whether drift exists. An LLM may later *explain* a finding (§8), but it never creates or closes one.
2. **Report, never correct.** Each finding names the file a human should change (§6). Falcon writes only its own report.
3. **Not evaluated is not OK.** A check whose live data is unavailable reports `not_evaluated` with the reason. It never reports "no drift".
4. **Stable findings.** The same drift on the next run is the same finding, with the same `id`. A finding is not repeated as new each time.
5. **Same identities as weaver.** Services, nodes and edges are matched with the normalisation in edges-spec §2.3 and §4.1.

2. Inputs

Input	Source	Used by
Service map	`weaver.nodes`, `weaver.edges` (MCP)	§4.2, §4.3
Workloads of each service	falcon's `service-mappings.yaml` (falcon-architecture §1)	all checks
Workloads, Services, HPAs, pod images and labels	kube-state-metrics series (`kube_deployment_*`, `kube_statefulset_*`, `kube_service_info`, `kube_horizontalpodautoscaler_info`, `kube_pod_container_info`, `kube_pod_labels`)	§4.1, §4.1
Observed calls	service-mesh and client metrics: whatever is exported (§4.2)	§4.2

Each run reads one consistent weaver build. Falcon records its build id (`weaverBuild`, weaver-architecture §2) in the report.

3. Run

```
falcon drift-detect domain <name> [--check mapping,dependencies,infrastructure] [--window 7d]
```

- **Default:** all checks run. `--window` is the observation window for §4.2, and defaults to 7 days.
- **When it runs:** on demand, on a schedule, and after every weaver build of the domain.
- **Output:** `drift/<domain>.yaml` (§5), replaced atomically on every run. Also a summary on stdout.
- **Exit codes:**
  - `0`: no open findings at `high` severity or above.
  - `1`: at least one such finding.
  - `2`: the run failed, for example weaver or the metrics backend was unreachable.

4. Checks

4.1 `mapping`: service-mappings.yaml vs cluster

Evaluated from kube-state-metrics series: an object exists when its series does.

Result	Condition	Severity
`missing_workload`	a listed `namespace/kind/name` does not exist	high
`missing_service` / `missing_hpa`	a listed Service or HPA does not exist	medium
`hpa_mismatch`	the HPA's `scaleTargetRef` is not the listed workload	medium
`unclaimed_workload`	a workload in a namespace that the mapping uses belongs to no `serviceName`	low
`unknown_service`	a `serviceName` is not a service in weaver's map	high

4.2 `dependencies`: code map vs observed calls

Observed calls are collected over `--window` from **metrics only**; falcon reads no traces or logs. Each metric source maps to an edge as follows:

Source	Gives `from → to kind resource` from
Istio `istio_requests_total`	`source_workload` / `source_workload_namespace` → `destination_workload` / `destination_workload_namespace`; `request_protocol` gives `rest` or `grpc`. Istio doesn't label the request path by default, so these edges match at `from → to kind` level
Istio `istio_tcp_sent_bytes_total`	`source_workload` → `destination_service`: TCP connections to brokers and databases, matched to infrastructure nodes through their `aliases`. No topic or table, so `from → to kind` level
Kafka client metrics, if exported	producer and consumer client ids per topic: the only source that gives a topic

Mapping to weaver's terms:

- Workloads become services via `service-mappings.yaml`.
- Hosts become nodes via node `aliases` (edges-spec §2.3).
- Resources are normalised as in edges-spec §3.1.
- A source that can't supply the resource matches at `from → to kind` level only. The finding says so.

Result	Condition	Severity
`unexpected`	observed, but not in `edges.yaml`: either weaver missed it, or the code calls something weaver can't see	high if `to` is outside the domain, medium otherwise
`dormant`	in `edges.yaml`, but not observed in the window	info. Low if its caller's `entries` were observed with traffic: the code path runs, yet the call was never seen
`criticality_conflict`	a `call: optional/async` edge whose failures, observed in the same window, coincide with errors on its caller's entry point	medium
`not_evaluated`	no source covers this edge's kind	info
`probable_answer`	an observed call matches no edge, but the same `from` has an edge of the same `kind` to `unknown/<question-id>` (edges-spec §2.1): the observation probably answers that question. Reported once, with the question id, instead of an `unexpected` plus a `dormant`	info

A `dormant` edge is never removed as dead. It may be a rarely used path, which is exactly what the static map is for.

4.3 `infrastructure`: node properties vs observed addresses

Only workloads with `env: production` in `service-mappings.yaml` (falcon-architecture §1.2) are compared: weaver's map describes production, so a staging namespace would always differ. Other workloads are `not_evaluated` here.

Falcon reads no configuration, pod specs or secrets; only metrics. Addresses come from client metrics that carry them as labels (for example Kafka client metrics with broker hosts, or connection-pool metrics with the database host), mapped to services by workload.

Result	Condition	Severity
`address_mismatch`	an observed address is not the node's address nor one of its `aliases`. Kafka brokers are compared as sets: one shared broker is enough	high
`extra_instance`	the workloads talk to an instance of this type that the map has no node for	medium
`not_evaluated`	no metric carries the address	info

Addresses are normalised as in edges-spec §2.3: lower-cased, scheme removed, default port added, broker lists sorted.

5. Report: `drift/<domain>.yaml`

```
version: falcon/drift/v1.0
domain: payments
run: {at: 2026-10-24T11:00:00Z, window: 7d, weaverBuild: 42}
summary: {open: 1, high: 1, medium: 0, low: 0, info: 0, not_evaluated: [dependencies/redis]}
findings:
  - id: 2b44f0d1
    check: dependencies
    result: unexpected
    severity: high
    subject: {edge: "checkout → external/api.fraudcheck.io rest POST /v2/score"}
    live: {source: istio_requests_total, calls: 18230, firstObserved: 2026-10-22T07:41:00Z}
    fix: {file: edges.override.yaml, action: "add the edge, or find why weaver missed it"}
    firstSeen: 2026-10-24T11:00:00Z
```

```
    lastSeen:  2026-10-24T11:00:00Z
    status: open
```

Field	Meaning
`id`	Stable: a hash of `check`, `result` and `subject`. The same drift keeps its `id`, `firstSeen` and `status` across runs
`fix`	The file to change and the action, never applied automatically (§6)
`status`	`open` · `accepted` (§7) · `resolved`. A finding no longer detected becomes `resolved` and is kept for 30 days

6. Where fixes go

Finding	File	Owner
`mapping` results	`service-mappings.yaml`	falcon operator
`unexpected`, `dormant`, `criticality_conflict`	`edges.override.yaml`, or a weaver extraction bug	service team
`address_mismatch`, `extra_instance`	`edges.override.yaml` (node `props`, `aliases`), or the deployment config	service team

7. Accepting drift

Known, intended drift is recorded in `drift.accept.yaml`, kept next to `service-mappings.yaml`:

```
version: falcon/drift.accept/v1.0
accepted:
  - id: 2b44f0d1
    reason: "fraudcheck called through a sidecar; not in code by design"
    by: payments-team
    until: 2027-01-31
```

- **Effect:** an accepted finding gets `status: accepted` and leaves the summary counts. It still appears in the report.
- **Expiry:** `until` is required. An expired entry turns the finding back into `open`.
- **Unknown ids:** an id that no longer matches a finding is reported as a warning.